



Packaging & AI Pair Operator

BUILD
REVIEW
PROVE

TAG 4 & 5

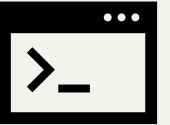
- > Persistenz Theorie
- > Fehler beweisen
- > Dev und Prod paketieren
- > Vibcode Your Zeiterfassungssystem
- > Nächstes Mal: Selbständiges Arbeiten Zeiterfassung VibeCodeTheWorld

ABLAUF

- > Persistenz
- > Troubleshooting A/B/C
- > Packaging
- > Hermes lokal einrichten
- > Zeiterfassung bauen und prüfen



Ein neuer Pod darf keine Arbeitszeit löschen



```
OHNE PVC
Pod weg → Daten weg

MIT PVC
Pod weg → Daten bleiben
```

WARUM PERSISTENZ?

- > Im Pod gespeicherte Daten verschwinden mit dem Pod.
- > Ein PVC verbindet die Anwendung mit dauerhaftem Speicher.
- > Die Zeiterfassung schreibt deshalb nach /data.

Man konfiguriert die Persistenz in einer eigenen pvc.yaml und bindet sie im deployment.yaml unter /data

```
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: zeiterfassung-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

```
# deployment.yaml – innerhalb von spec.template.spec
containers:
  - name: zeiterfassung
    image: zeiterfassung:1.0
    volumeMounts:
      - name: daten
        mountPath: /data

volumes:
  - name: daten
    persistentVolumeClaim:
      claimName: zeiterfassung-data
```

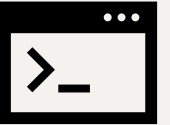
/data ist der Ordner **im Container**; alles, was die App dort speichert, wird durch den PVC ausserhalb des Pods dauerhaft abgelegt – den tatsächlichen Speicherort auf dem Minikube-Host verwaltet Kubernetes automatisch.

Zum Beispiel: Einstempeln, Kubernetes ersetzt den Pod nach einem Absturz, und dank PVC bleibt der gespeicherte Arbeitsbeginn weiterhin vorhanden.

Die Anwendung speichert beispielsweise /data/times.jsonl; der PVC sorgt dafür, dass diese Datei auch nach einem Pod-Neustart erhalten bleibt.



Ein neuer Pod darf keine Arbeitszeit löschen

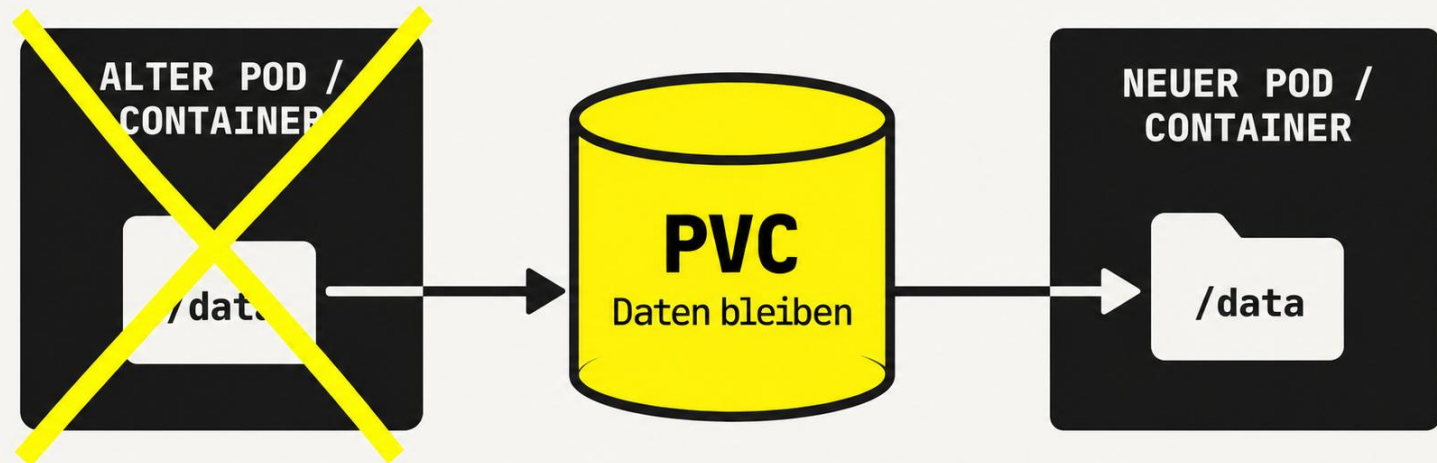


OHNE PVC
Pod weg → Daten weg

MIT PVC
Pod weg → Daten bleiben

WARUM PERSISTENZ?

- > Im Pod gespeicherte Daten verschwinden mit dem Pod.
- > Ein PVC verbindet die Anwendung mit dauerhaftem Speicher.
- > Die Zeiterfassung schreibt deshalb nach /data.

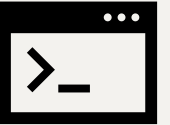


Container weg – PVC bleibt – Daten wieder eingebunden

Der PVC fordert Speicher an; die Daten liegen im PV auf dem Minikube-Node.



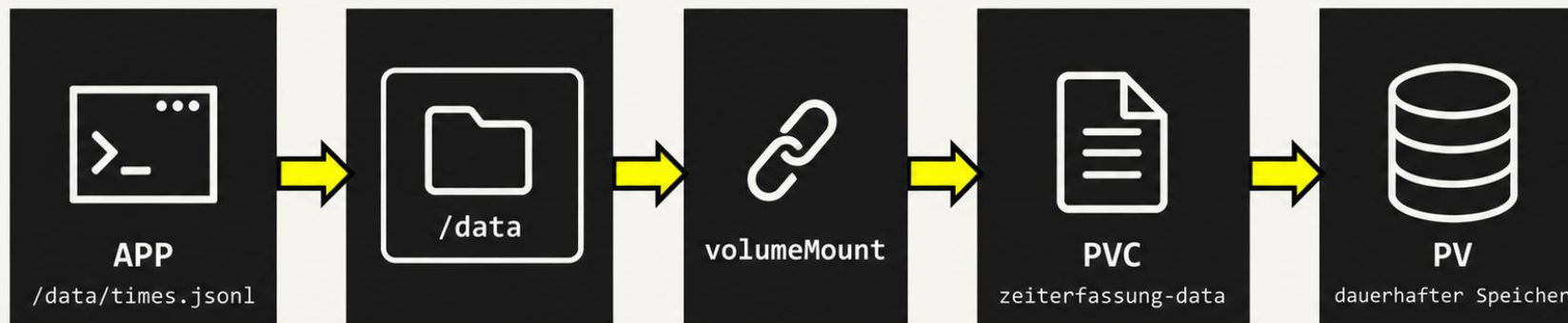
Persistenz braucht drei Verbindungen



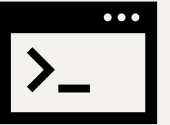
```
App
/data/times.jsonl
↓
volumeMount: /data
↓
PVC: zeiterfassung-data
```

APP → MOUNT → PVC

- > pvc.yaml fordert Speicher an.
- > deployment.yaml bindet den Claim als Volume ein.
- > volumeMount stellt ihn im Container unter /data bereit.



Fehler sehen ist nicht Fehler verstehen



TROUBLESHOOTING

mit get, describe und logs die Ursache prüfen

CrashLoopBackOff bedeutet, dass ein Container wiederholt abstürzt und Kubernetes vor jedem weiteren Neustart zunehmend länger wartet.

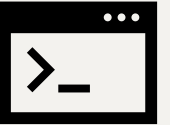
Zum Beispiel bei einem CrashLoopBackOff:

```
2026-09-11 10:14:02 Starting Zeiterfassung...
2026-09-11 10:14:02 ERROR: Permission denied: /data/times.jsonl
2026-09-11 10:14:02 Application terminated with exit code 1
```

Die App stürzt ab, weil sie nicht in /data/times.jsonl schreiben darf.



Drei Gruppen, drei echte Fehlerbilder



```
logs --previous  
describe + Events  
svc + endpoints
```

> CrashLoopBackOff

Der Container startet, stürzt wiederholt ab und Kubernetes wartet zwischen den Neustartversuchen immer länger.

> ImagePullBackOff

Kubernetes kann das angegebene Container-Image nicht herunterladen, beispielsweise wegen eines falschen Namens oder fehlender Berechtigung.

> falscher Service-Port

Der Service leitet Anfragen an einen falschen targetPort weiter und erreicht deshalb die Anwendung im Pod nicht.



Your Turn: Deployment und Service bis zum Browser-Test



APPLY

Manifeste
anwenden

Deployment + Service

OBSERVE

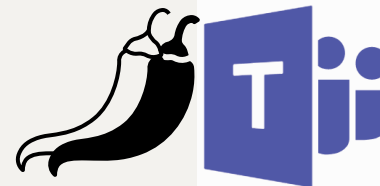
Zustand lesen

Pods + Rollouts

FIX

Unbekannter
Fehler
lösen

Beweis statt Raten



Aufgaben

Abgabe: laufender Workload plus INC-Protokoll



Ziel: Anwendung unter `http://192.168.110.60:8080` erreichbar machen.

Ihr erhaltet pro Gruppe 1 ZIP

- Kubernetes_Troubleshooting_Variante_A_new.zip
- Kubernetes_Troubleshooting_Variante_A_new.zip
- Kubernetes_Troubleshooting_Variante_A_new.zip

Kubernetes_Troubleshooting_Variante_[X]_new.zip
Auf LM nachbilden (eventuell altes K8S löschen)

zeiterfassung/
/app
 .dockerignore
 app.py
 Dockerfile

/k8s
 deployment.yaml
 service.yaml

Abgabe & HowTo

GruppenauftragKubernetes_Tag4_AbgabeVorlage.docx

In definierten Gruppen

1h

- 1) Image bauen
- 2) Image verwenden im Minikube Cluster
- 3) Testen `http://192.168.110.60:8080`
- 4) Kubernetes logs beobachten
- 5) fixen



Aufgaben

Gruppenauftrag 26A

Gruppe1

Arno E.

Alwin L.

Gibethra K.

Variante A

Gruppe2

Felek S.

Milan T.

Marco S.

Variante B

Gruppe3

Ruben F.

Kiril R.

Daniel E.

Variante C

Gruppe4

Sebastian R.

Marcel Z.

Nico J.

Variante A

Gruppe5

Yannick Merz

Leon T.

Luigi M.

Variante B

Gruppe6

Yogani Y.

Luca G.

Yannick Minder

Variante C



Gruppenauftrag 26C

Gruppe1

Noel E.

Mike R.

Seya S.

Variante A

Gruppe2

Besart S.

Kevin A.

Andre T.

Variante B

Gruppe3

Kai K.

Marek H.

Alec B.

Aro I.

Variante C

Gruppe4

Boris K.

Mael C.

Sven S.

Variante A

Gruppe5

Nuno G. ?

Praveen M.

Variante B

Gruppe6

David S.

Ben B.

Leila Y.

Sean P.

Variante C





LÖSUNGEN



3 Fehlbilder brauchen unterschiedliche Beweise



CRASHLOOPBACKOFF

App beendet
sich

logs --previous +
describe

IMAGEPULLBACKOFF

Image nicht
verfügbar

describe +
Events

SERVICE-PORT

Service
erreicht
Pods nicht

Service, Labels,
targetPort



Aufgaben





PAUSE 30Min

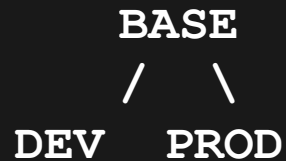
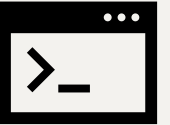




Dev und Prod paketieren



Eine Basis, zwei Umgebungen



PACKAGING

- > Gemeinsames bleibt in der Base; nur Unterschiede gehören in Dev und Prod.

Beim Packaging enthält die **Base** alle gemeinsamen Kubernetes-Manifeste, während **Dev** und **Prod** nur ihre jeweiligen Abweichungen ergänzen.

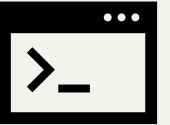




Paketieren – auf was achten?



Kustomize hält Unterschiede klein



```
base/  
  deployment.yaml  
  service.yaml  
overlays/  
  dev/kustomization.yaml  
  prod/kustomization.yaml
```

ORDNER STATT COPY-PASTE

- > Base enthält gemeinsame Manifeste.
- > Overlays ändern nur umgebungsspezifische Werte.
- > kubectl kann das Ergebnis direkt rendern und anwenden.

Ressourcen:

base/kustomization.yaml
die Unterschiede jeweils in
overlays/dev/kustomization.yaml & overlays/prod/kustomization.yaml

base/kustomization.yaml

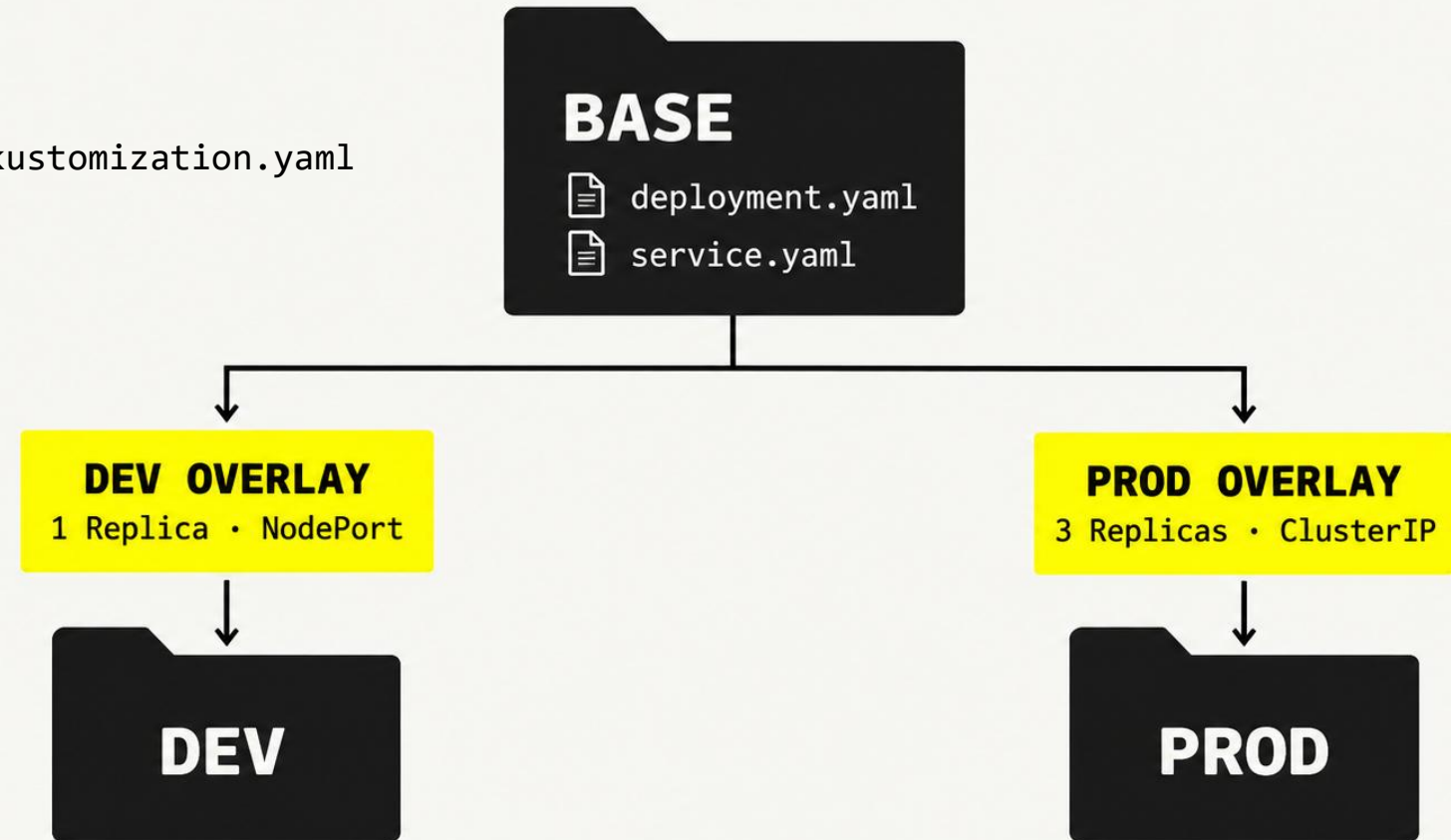
```
resources:  
- deployment.yaml  
- service.yaml
```

overlays/dev/kustomization.yaml

```
resources:  
- ../../base  
replicas:  
- name: zeiterfassung  
  count: 1
```

overlays/prod/kustomization.yaml

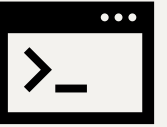
```
resources:  
- ../../base  
replicas:  
- name: zeiterfassung count: 3
```



Gemeinsame Base + kleine Overlays = fertige Umgebungen



Dev und Prod unterscheiden sich absichtlich



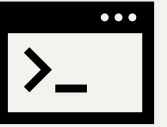
DEV	PROD
1 Replica	3 Replicas
NodePort	ClusterIP
lokales Image	Registry-Image

GLEICHE APP • ANDERE REGELN

- > Dev: schnell testen und lokal erreichen.
- > Prod: robust, begrenzt und kontrolliert exponiert.



Kustomize



```
KUSTOMIZE  
kubectl apply -k overlays/dev
```

```
HELM  
helm upgrade --install ...
```

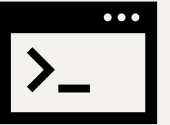
HELM IST DIE ERWEITERUNG

- > Kustomize: normales YAML plus Patches; in kubectl integriert.
- > Helm: Templates plus Values; geeignet für verteilbare Pakete.

Helm ist ein Paketmanager für Kubernetes, der zusammengehörige Manifeste als sogenannte Charts bündelt und über anpassbare values.yaml-Dateien installiert und konfiguriert.



Yaml yaml yaml



Datei	Aufgabe
deployment.yaml	Startet die Pods und definiert Image, Replikate, Ports und Volumes
service.yaml	Macht die Pods im Netzwerk erreichbar
pvc.yaml	Fordert dauerhaften Speicher für die Daten an
kustomization.yaml	Verknüpft diese Dateien und definiert Dev-/Prod-Abweichungen
values.yaml	Gibt es nur bei Helm und enthält einstellbare Werte
Helm-Templates	Sind Vorlagen für Deployment, Service und PVC

Kustomize sieht so aus:

```
base/
├── deployment.yaml
├── service.yaml
├── pvc.yaml
└── kustomization.yaml

overlays/
├── dev/
│   └── kustomization.yaml
└── prod/
    └── kustomization.yaml
```



VibeCode the World (Mit Schlaunen Agenten)



HERMES AGENTS kurz erklärt + unser gewolltes Setup für eine sauber laufende Zeiterfassung





Unterschied VibeCoding & Agents?



Unterschied VibeCoding & Agents



VibeCoding

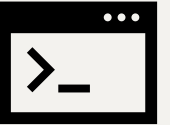
Definition: Ein Trend in der Softwareentwicklung, bei dem Sie nicht mehr selbst Code tippen, sondern der KI in natürlicher Sprache sagen, was sie tun soll.

KI Agents

Definition: Intelligente Programme, die mit einem Ziel vor Augen autonom handeln, sich erinnern und Werkzeuge (wie APIs oder Datenbanken) einsetzen.



Hermes verbindet ein Modell mit Werkzeugen



HERMES AGENT

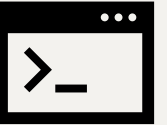
- | lokales Modell
- | Dateien
- | Terminal

WAS LÄUFT WO?

- > Hermes steuert den Arbeitsablauf auf der Ubuntu-VM.
- > Das Sprachmodell liefert Vorschläge und Entscheidungen.
- > Terminal- und Dateiwerkzeuge führen erlaubte Schritte lokal aus.



Die Installation braucht nur wenige Pakete



```
sudo apt update
sudo apt install -y git curl xz-utils

curl -fsSL https://hermes-
agent.nousresearch.com/install.sh | bash

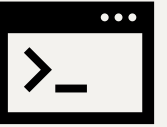
source ~/.bashrc
```

AUF DER UBUNTU-VM

- > Docker install
- > Das Installationsskript richtet Hermes im Benutzerprofil ein.
- > Nachher Shell neu laden oder Terminal neu öffnen



Für das Lab wählen wir den kostenlosen lokalen Weg



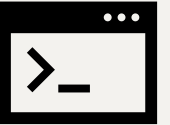
```
Hermes
  ↓ Custom Endpoint
Ollama auf der VM
  ↓
lokales Modell

Messaging-Integrationen: überspringen
```

- > Ollama führt das Modell direkt auf der VM aus
- > Im Setup: Full Setup oder Blank Slate, danach Custom Endpoint.



Das lokale Modell muss zur Hardware passen



```
curl -fsSL https://ollama.com/install.sh | sh

ollama --version

# leicht, Chat
ollama pull llama3.2:3b

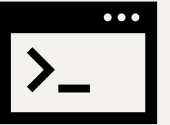
# Tool Calling, 24+ GB RAM
ollama pull gemma4:31b
```

OLLAMA INSTALLIEREN

- > Ein kleines Modell funktioniert auf schwachen VMs, kann aber meist keine Werkzeuge aufrufen.
- > Für zuverlässige Datei- und Terminalaktionen empfiehlt Hermes ein Modell mit Tool Calling.
- > Vor dem Lab RAM und freien Speicher prüfen.



Hermes spricht über einen lokalen Endpunkt mit Ollama



```
hermes setup
```

Provider	Custom Endpoint
Base URL	http://localhost:11434/v1
API Key	no-key
Model	gemma4:31b

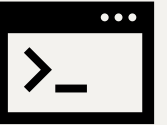
```
# keine Portal-Anmeldung nötig
```

HERMES SETUP

- > Provider: Custom Endpoint
- > Base URL: http://localhost:11434/v1
- > API Key: leer lassen oder no-key
- > Model: exakt der mit Ollama geladene Name



Drei Tests zeigen, ob der lokale Aufbau funktioniert



```
curl http://localhost:11434/api/tags
hermes --version
hermes
```

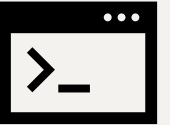
```
Test im Chat:
"Liste die Dateien im aktuellen Ordner.
Ändere noch nichts."
```

ERST PRÜFEN, DANN BAUEN

- > Ollama antwortet auf Port 11434
- > Hermes erkennt Version und Provider
- > Ein Testauftrag zeigt, ob Dateiwerkzeuge funktionieren



Der Arbeitsordner begrenzt den Auftrag



ERLAUBT

- ✓ Projektdateien bearbeiten
- ✓ Image bauen und lokal testen
- ✓ Manifeste rendern

NUR NACH RÜCKFRAGE

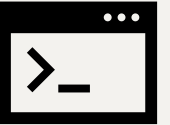
- ! sudo ! rm ! firewall
- ! externe Veröffentlichung

SICHERHEITSGRENZEN

- > Hermes arbeitet nur unter ~/workspace/zeiterfassung
- > Sudo, Firewall, Paketinstallation und Löschbefehle brauchen Zustimmung
- > Passwörter und Tokens gehören weder ins Prompt noch ins Git-Repository



Prompt 1 lässt Hermes zuerst planen



```
Analysiere den Ordner zeiterfassung.  
Plane eine kleine Flask-Zeiterfassung  
mit EIN, PAUSE, ZURÜCK und AUS.
```

```
Daten müssen unter /data erhalten bleiben.  
Plane Dockerfile, /health, PVC sowie  
Kustomize-Overlays für dev und prod.
```

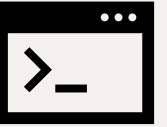
```
Ändere noch keine Datei.
```

NOCH KEINE DATEIEN ÄNDERN

- > Der erste Prompt erzeugt Architektur, Dateiliste und Testplan
- > Prüfen Persistenz, Exposition und Secrets
- > Erst nach dem Review folgt die Umsetzung



Prompt 2 setzt nur den geprüften Plan um



```
Setze den freigegebenen Plan um.  
Arbeite nur im aktuellen Projektordner.  
Zeige vor jeder Änderung den Diff.
```

```
Frage vor sudo, Löschen, Firewall oder  
externer Veröffentlichung nach.  
Teste Build, /health, Persistenz und  
die gerenderten dev/prod-Manifeste.
```

IMPLEMENTIEREN UND NACHWEISEN

- > Hermes arbeitet schrittweise und zeigt jede Änderung
- > Nach jedem Abschnitt folgt ein konkreter Test



Arbeitsauftrag ab/bis nächstes mal



Dieses Mal/Nächstes Mal:

Selbstständiges Arbeiten Zeiterfassung VibeCodeTheWorld

Zeiterfassungs-Service VibeCoden mit Hermes

Selbstständiges erarbeiten

Abgabe bis 18.09.2026 – 18.45Uhr

Mindestanforderungen:

- (Cyber)security hintergedanken
- Automatische HealthCheck's
- Einstempeln & Ausstempeln
- Freiwillig: Userbasiert / RBAC (Theorie (über)nächstes mal)
- Freiwillig: Raport/Notiz unter jeder «Stempelung» Bsp. «Support KYZ Kunde Meyer»

Abgaben:

- Video mit
 - Zeiterfassung vorführen
 - Max 2.5 Minuten
- Dokument mit
 - SOLL – was willst Du machen
 - Auflistung Security-Anforderungen
 - Healthchecks
 - Max 4 Seiten A4

